CS 301 · MEETING 21

Data and validation

A prediction task, a baseline, and an honest comparison

A model takes an input and produces a prediction. To decide whether that prediction is useful, we need a precise task, a meaningful comparison, and observations that were not used to fit or select the model.

Our running example is a daily bike-rental forecast. At the start of a day, we predict the total number of rentals during that day. We assume the preceding day's total has been finalized and is available. The historical data are Capital Bikeshare records from 2011–2012, distributed by the UCI Machine Learning Repository.

> **The essential distinction.** A retrospective record tells us what eventually happened. A predictor may use only the part of that record that would have been available when the prediction was needed.

What you should be able to do. Specify a target and prediction time; identify allowed inputs; explain training, validation, and test roles; fit and evaluate a baseline; calculate MAE and MSE; and recognize leakage.

---

1. Define the task before selecting columns

One row represents a calendar day. The target `cnt` is the total rentals that day. We use two input features: `yesterday_rentals`, the previous day's total, and `workingday`, a calendar flag known in advance. The feature matrix **X** has one row per example and one column per feature; the target vector **y** contains the corresponding observed answers.

Today's final casual-rider and registered-rider counts are unavailable at the start of the day. The observed daily weather is also unavailable then. An archived weather forecast issued beforehand could be usable, but it is different data from the observed daily weather in this file.

Regression predicts a numerical target, such as a count. Classification predicts a category, such as whether the count exceeds 5,000. These questions need different evaluation measures. We follow the numerical task here.

---

2. Fit on earlier observations and evaluate on later ones

Sorting the 731 source dates and adding yesterday's count leaves 730 examples: the first day has no preceding count in the file. We use 364 days from 2011 for training, 182 days from January–June 2012 for validation, and reserve 184 days from July–December 2012 for final testing.



Split first. Fit only on the training rows.

Time moves from left to right. Fit on the earlier rows; retain the final period while choosing a model.

Dataset	Job	What it influences
Training	Fit the model	Learned weights, rules, or stored quantities
Validation	Choose a candidate	Features, model family, and settings
Test	Assess the fixed procedure	The final reported evaluation

Suppose we need a forecast at the start of January 2. A shuffled split could fit on January 3 outcomes while holding out January 2. That fitted model could not have existed when the forecast was needed. This is why the ordering matters.

For this task, time order matters: we want evidence about prediction after the training period. Randomly mixing dates answers a different question. A random split can be appropriate for independent examples from the same population; it is not a universal evaluation rule.

The validation experiment makes successive one-day-ahead predictions using a fixed fitted model. After a validation day ends, its count is available as the next day's lagged input. The model is not refitted. This is different from forecasting six months ahead without observing intervening days.

Validation results have influenced development. If we use test results repeatedly to choose models, the test set also becomes part of development. Keep it reserved until choices are fixed. No test outcomes are scored in this meeting.

3. Establish a baseline

A mean baseline predicts the average training target for every input. It ignores the features but still learns one quantity from training data. A predictor that uses features should be compared against this simple alternative.

$$\hat{y} = \text{mean}(y_{\text{train}})$$

For the four training targets 2, 4, 6, 8, a constant prediction c has mean squared error $5 + (c - 5)^2$. The minimum occurs at $c = 5$, the training mean. The median instead minimizes mean absolute error among constant predictions.

For a time series, another useful baseline is persistence: predict that today's count equals yesterday's. It has no fitted parameters and uses a recent observed quantity. A feature-independent baseline alone can be too easy to beat.

---

4. Measure the size of the mistakes

Suppose one forecast is 8 rentals and the observed count is 10. Another forecast is 22 and the count is 20. One misses by 2 below the outcome; the other misses by 2 above it. The signed mistakes cancel even though both forecasts are wrong.

A residual is the observed value minus the prediction: $e = y - \hat{y}$. Positive and negative residuals can cancel. For observations 10, 20, 30, 40 and predictions 8, 22, 26, 44, the residuals are 2, −2, 4, −4. Their average is zero although every prediction is wrong.

$$\text{MAE} = \text{mean}(|y - \hat{y}|)$$
$$\text{MSE} = \text{mean}((y - \hat{y})^2)$$

The example has MAE = 3 and MSE = 10. MAE uses the target's units. MSE uses squared target units and gives greater weight to large errors. A model comparison should name the metric and its units; an unspecified "error" is incomplete.

---

5. Fit, predict, measure

```
model = DummyRegressor(strategy="mean")
model.fit(X_train, y_train)
y_pred = model.predict(X_valid)
mae = mean_absolute_error(y_valid, y_pred)
```

`fit` learns and stores model state. `predict` receives new inputs and returns predictions. The validation targets enter only when we evaluate those predictions. They are not arguments to fitting or prediction.

Different predictors can share this interface while retaining different information. Here the baseline retains a constant; the linear model learns coefficients for yesterday's count and the working-day flag. In both cases we use the same split and evaluation procedure.

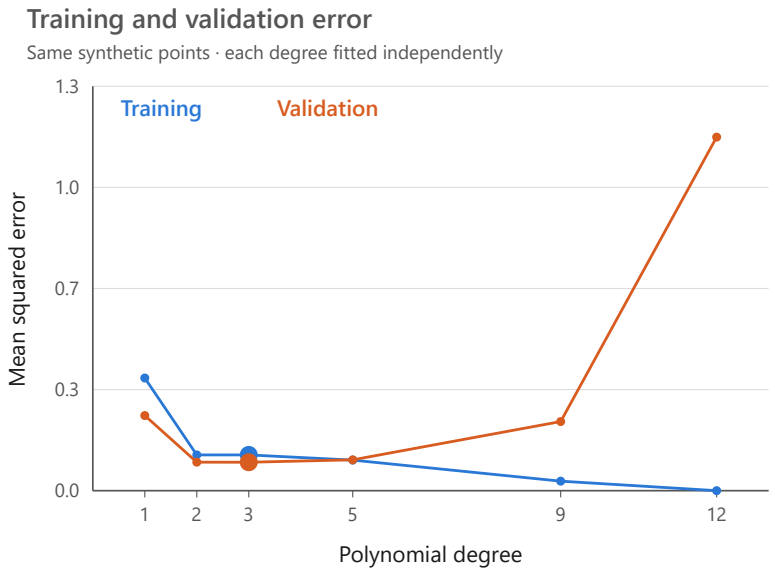
Predictor	Validation MAE (rentals)	Validation MSE (rentals^2)
Training mean	2,048.4	5,978,764.0
Yesterday's count	900.6	1,621,703.7
Fitted linear model	936.7	1,520,025.8

Yesterday's count wins on MAE; the linear model wins on MSE. Both improve on the training mean in this period. This is evidence about these predictors under this task, split, and metric. It does not establish a universal ranking or a causal explanation for rental demand.

6. Flexibility and leakage

Imagine measuring a controlled temperature of 20°C twice. The sensor reads 20.2°C, then 19.8°C. A rule that passes exactly through the first reading misses the repeated reading by 0.4°C. An exact fit to an observation can include measurement noise.

A flexible model can follow noise in its training examples. In the separate synthetic curve experiment, a degree-12 polynomial nearly interpolates the 14 training points, but predicts the independent validation observations less accurately than a modest-degree curve. Increasing flexibility is a choice to evaluate, not a guarantee of better predictions.



A separate synthetic experiment: training error falls with flexibility, while validation error eventually rises.

The completed January 1, 2012 report lists 686 casual rentals and 1,608 registered rentals. Their sum is exactly 2,294, the observed total. But at the start of January 1 those completed counts did not exist. A zero-error reconstruction of the finished day would not have supplied a forecast.

Leakage occurs when model development uses information that would not be available in the intended prediction setting. The identity `casual + registered = cnt` has zero reconstruction error, but both components arrive after the day. Even a chronological split cannot make them available beforehand.

Consider training ride distances 2, 4, 6, and one missing value. Replacing the missing distance with the observed training mean gives 4 km. If a validation ride of 100 km enters the average, the replacement becomes 28 km. Validation data would then change a training input. These distances are invented; the bike file has no missing entries.

Preprocessing can also learn from data. A scaler or imputer must learn its quantities from training rows and apply that same transformation to later rows. During cross-validation, learned preprocessing belongs inside each training fold. A pipeline helps organize this correctly.

Imagine predicting a runner's pace from workout records. If four of Nora's workouts were used for fitting, her fifth is a new row from a familiar runner. To assess prediction for a runner absent from training, keep all of Nora's workouts together in validation.

The required novelty also matters. If the task is prediction for a new person, all readings from that person should be held out together. Splitting individual readings can leave the same person in both training and validation and answer a different question.

---

7. State a result another person can assess

A useful result identifies the prediction target and time, the available inputs, the training and validation periods, the baseline, the metric and units, and the measured comparison. It also states a limitation and a next experiment that preserves the final test set.

Check your understanding. Explain why a zero-error reconstruction may fail as a forecast. Explain why selecting the best test score uses the test set for development. Explain what a fitted mean baseline needs to predict for a new row.

Sources

Fanaee-T, H. (2013). *Bike Sharing*. UCI Machine Learning Repository. doi:10.24432/C5W894. CC BY 4.0. Daily records are preserved; the teaching preparation adds a one-day lag.

- Cross-validation and evaluation
- Leakage and preprocessing
- DummyRegressor
- Regression metrics